DESENVOLVIMENTO MOBILE · MÓDULO 16 · COMUM ÀS TRILHAS · NÍVEL AVANÇADO

Backend Terceirizado (BaaS)

Computação em nuvem, modelos de serviço, *serverless*, panorama de Firebase, Supabase, AWS Amplify, Appwrite, PocketBase e Parse, prática com Supabase e critérios de escolha

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o papel do backend e os modelos de serviço em nuvem (IaaS, PaaS, SaaS, BaaS, FaaS).
- Comparar os principais BaaS do mercado quanto a dados, preço, licença e aprisionamento.
- Construir o backend do LabMonitor no Supabase: tabelas, RLS, tempo real, arquivos e funções.
- Consumir o Supabase em Kotlin e em Dart.
- Escolher um provedor com critérios técnicos, de custo e de LGPD.

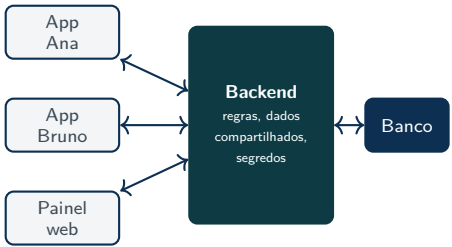
Roteiro

Neste módulo

- Fundamentos: backend e nuvem
- Panorama dos principais BaaS
- Supabase na prática
- Outros provedores
- Escolhendo e arquitetando
- Fechamento

Fundamentos: backend e nuvem

Por que o app precisa de um backend



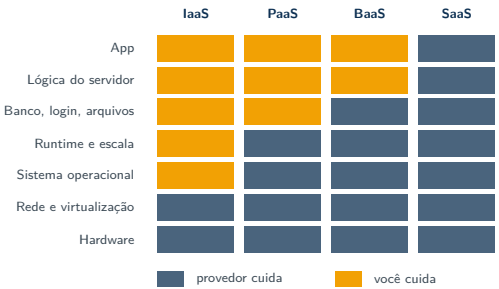
Backend

Parte do sistema que roda em servidores: guarda os dados compartilhados, autentica usuários, aplica regras de negócio, integra serviços e envia notificações.

O que o app sozinho não resolve

- **Compartilhar**: a reserva da Ana precisa aparecer para o Bruno.
- **Confiar**: o app pode ser modificado; regras importantes ficam no servidor.
- **Guardar segredos**: chaves de APIs pagas e credenciais.
- **Trabalhar com o app fechado**: alertas e relatórios agendados.

Modelos de serviço em nuvem



Modelo	Você recebe
IaaS	máquinas virtuais (EC2, Compute Engine)
PaaS	plataforma para rodar seu código (Cloud Run, Heroku)
FaaS	funções que rodam por evento (Lambda, Cloud Functions)
BaaS	backend pronto: banco, login, arquivos, tempo real
SaaS	software pronto para usar (Gmail, Notion)

Analogia

Pizza: fazer em casa (servidor próprio), comprar congelada (IaaS), pedir entrega (BaaS) ou comer no restaurante (SaaS).

Serverless e o modelo de custo

Serverless

Modelo em que o provedor aloca, escala e desliga os servidores automaticamente; você paga pelo **uso** (requisições, tempo de execução, leituras) e não por máquinas ligadas.

Partida a frio

Sem tráfego, a função "dorme". A primeira chamada depois disso precisa iniciar o ambiente, o que leva de centenas de milissegundos a alguns segundos.

Cobrança por	Exemplo de armadilha
leituras de documentos	tela que escuta uma coleção inteira em tempo real
tráfego de saída	fotos grandes baixadas a cada abertura
usuários ativos (MAU)	contas de teste esquecidas
invocações de função	laço que chama a função a cada segundo

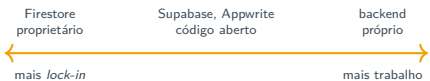
Atenção

Configure **alertas de orçamento** e limites antes de abrir o app ao público. Um defeito pode virar uma conta alta em horas.

Aprisionamento tecnológico (*vendor lock-in*)

Lock-in

Dependência de um fornecedor tão grande que trocar de serviço fica caro: dados em formato proprietário, APIs exclusivas, regras espalhadas pelo app.



Como reduzir o risco

- Esconder o provedor atrás de **repositórios** (Módulo 08): só a camada de dados conhece o SDK.
- Preferir padrões abertos: SQL, REST, JWT, S3.
- Exportar os dados periodicamente.
- Saber se é possível **auto-hospedar** o serviço.

Dica

Lock-in não é sempre ruim: é o preço de ganhar velocidade. O problema é aceitá-lo sem perceber.

LGPD e serviços de terceiros

Controlador e operador (LGPD)

O **controlador** decide por que e como os dados pessoais são tratados (a instituição dona do app). O **operador** trata os dados em nome do controlador (o provedor de BaaS).

Transferência internacional

Se os servidores ficam fora do Brasil, há transferência internacional de dados, que a LGPD só permite em hipóteses previstas (como cláusulas contratuais adequadas).

Antes de escolher

- Há região de dados no Brasil (São Paulo)?
- Existe contrato de tratamento de dados?
- Como exportar e apagar os dados de um usuário?
- Onde ficam logs e cópias de segurança?

Atenção

Não é assessoria jurídica: em projetos institucionais, envolva o encarregado de dados (DPO).

Panorama dos principais BaaS

Os principais provedores

Serviço	Dados	Código	Auto-hospedar	Destaque
Firebase	Firestore (documentos), Realtime Database; Data Connect (PostgreSQL)	proprietário	não	integração com Google, FCM, Crashlytics, offline nativo
Supabase	PostgreSQL	aberto	sim (Docker)	SQL, RLS, API REST automática, tempo real, Edge Functions
AWS Amplify	DynamoDB via GraphQL (AppSync)	proprietário	não	backend definido em TypeScript, ecossistema AWS
Appwrite	tabelas com permissões por linha	aberto	sim (Docker)	funções em várias linguagens, mensagens, painel completo
PocketBase	SQLite num único executável	aberto	só auto-hospedado	simplíssimo; roda até num Raspberry Pi
Parse (Back4App)	MongoDB ou PostgreSQL	aberto	sim	veterano; Back4App oferece a versão hospedada

Um pouco de história

O Parse foi comprado pelo Facebook em 2013, teve o código aberto e o serviço encerrado em 2017; a comunidade o mantém. Todos os serviços da tabela têm SDKs para Kotlin e Flutter.

Documentos ou tabelas?

	Documentos (Firestore)	Relacional (Postgres)
Estrutura	flexível, aninhada	tabelas com esquema
Relações	duplicação de dados	chaves estrangeiras e `JOIN`
Consultas	por índice, simples	SQL completo, agregações
Integridade	regras no app e nas *rules*	restrições no banco
Escala	horizontal automática	vertical, réplicas
Offline	embutido no SDK	feito no app (Módulo 10)

Como decidir

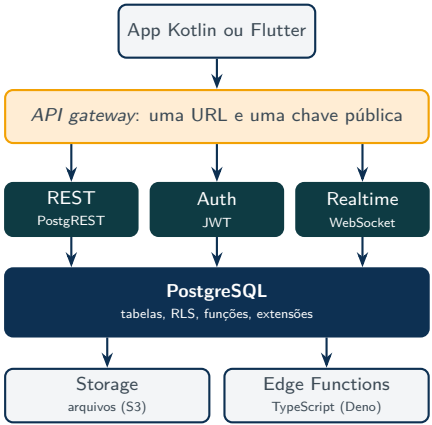
Dados muito relacionados (reservas, usuários, bancadas, relatórios) favorecem o relacional. Dados que variam muito de forma, lidos em blocos, favorecem documentos.

Teorema CAP

Com falha de rede num sistema distribuído, escolhe-se entre **consistência** (todos veem o mesmo dado) e **disponibilidade** (todos recebem resposta).

Supabase na prática

Arquitetura do Supabase



A ideia central

Tudo gira em torno de um PostgreSQL de verdade. A API REST é gerada a partir das tabelas; a autenticação gera JWTs; a segurança é feita **no banco**, com RLS.

Dica

Ambiente local completo com a CLI: `supabase start` sobe tudo em Docker, e `supabase db diff` gera migrações.

Chaves

A chave **pública** (*publishable*, antiga *anon*) vai no app. A chave **secreta** (*secret*, antiga *service_role*) ignora o RLS e nunca sai do servidor.

Tabelas e integridade no banco

supabase/migrations/001_reservas.sql

```
create table public.bancadas (
  id bigint generated always as identity primary key,
  codigo text not null unique,
  laboratorio text not null,
  livre boolean not null default true
);
create table public.reservas (
  id bigint generated always as identity primary key,
  bancada_id bigint not null references public.bancadas(id),
  usuario_id uuid not null default auth.uid()
             references auth.users(id),
  inicio timestamptz not null,
  fim    timestamptz not null,
  check (fim > inicio)
);
create extension if not exists btree_gist;
alter table public.reservas add constraint sem_sobreposicao
  exclude using gist (bancada_id with =,
                      tstzrange(inicio, fim) with &&);
```

Regra de negócio no lugar certo

A restrição de exclusão impede que duas reservas da mesma bancada se **sobreponham** no tempo (`&&` é o operador de sobreposição de intervalos). Nenhum app, nem com defeito, consegue violá-la.

Dica

No Firestore (Módulo 12), a mesma regra exigiu uma transação no app. Migrações SQL versionadas entram no Git.

Segurança em nível de linha (RLS)

supabase/migrations/002_politicas.sql

```sql
alter table public.reservas enable row level security;

create policy "todos logados veem as reservas"
  on public.reservas for select to authenticated
  using (true);

create policy "cada um cria a sua"
  on public.reservas for insert to authenticated
  with check (usuario_id = (select auth.uid()));

create policy "cada um cancela a sua"
  on public.reservas for delete to authenticated
  using (usuario_id = (select auth.uid()));
```

RLS — *Row Level Security*

O PostgreSQL acrescenta a cada consulta uma condição por linha baseada no usuário. `using` filtra as linhas visíveis; `with check` valida as gravadas.

Como o banco sabe quem é

O app envia o JWT do login; o usuário chega ao SQL por `auth.uid()`.

O erro mais grave

Tabela exposta **sem** RLS: qualquer pessoa com a chave pública lê e altera tudo.

Cliente Dart (Flutter)

supabase_flutter

```dart
await Supabase.initialize(url: supabaseUrl,
    anonKey: chavePublica);          // chave publishable
final db = Supabase.instance.client;

await db.auth.signInWithPassword(email: email, password: senha);

final livres = await db.from('bancadas')
    .select('id, codigo, laboratorio')
    .eq('livre', true)
    .order('codigo');

try {
  await db.from('reservas').insert({
    'bancada_id': 7,
    'inicio': inicio.toUtc().toIso8601String(),
    'fim': fim.toUtc().toIso8601String(),
  });
} on PostgrestException catch (e) {
  if (e.code == '23P01') mostrar('Horário indisponível');
}
```

Construtor de consultas

`select`, `eq`, `order` viram uma requisição REST que o PostgREST traduz em SQL. O RLS é aplicado em todas.

Código 23P01

Código do PostgreSQL para violação de restrição de exclusão: a sobreposição foi barrada pelo banco e vira mensagem amigável no app.

Dica

Coloque essas chamadas num `ReservaRepositorySupabase` (Módulo 08).

Cliente Kotlin (Android)

supabase-kt

```
val db = createSupabaseClient(supabaseUrl, chavePublica) {
  install(Auth)
  install(Postgrest)
  install(Realtime)
}

db.auth.signInWith(Email) {
  email = emailDigitado
  password = senhaDigitada
}

val livres = db.from("bancadas").select(
    columns = Columns.list("id", "codigo", "laboratorio")
) {
  filter { eq("livre", true) }
  order("codigo", Order.ASCENDING)
}.decodeList<Bancada>()

db.from("reservas").insert(NovaReserva(7, inicio, fim))
```

Mesmo modelo mental

O SDK Kotlin usa *plugins* instalados no cliente e *kotlinx.serialization* para converter as linhas em classes `@Serializable`.

Dica

As chamadas são funções `suspend`: rode-as no `viewModelScope` (Módulo 06).

Atenção

Nunca coloque a chave secreta no `BuildConfig`: o APK pode ser descompactado.

Tempo real

Dart: lista sempre atualizada

```dart
final reservasDaB7 = db
    .from('reservas')
    .stream(primaryKey: ['id'])
    .eq('bancada_id', 7)
    .order('inicio');
// StreamBuilder ou view model
```

Habilitar a tabela no Realtime

```sql
alter publication supabase_realtime
  add table public.reservas;
```

Kotlin: mudanças como Flow

```kotlin
val canal = db.channel("reservas")
val mudancas = canal
  .postgresChangeFlow<PostgresAction>(
      schema = "public") {
    table = "reservas"
  }
canal.subscribe()
mudancas.collect { acao -> atualizar(acao) }
```

Como funciona

O serviço Realtime acompanha o registro de alterações do PostgreSQL (replicação lógica) e envia as mudanças por WebSocket, respeitando o RLS.

Arquivos e Edge Functions

Foto do equipamento (Dart)

```dart
await db.storage.from('equipamentos').upload(
    'eq-42.jpg', File(foto.path));
final url = db.storage.from('equipamentos')
    .getPublicUrl('eq-42.jpg');
```

Kotlin

```kotlin
db.storage.from("equipamentos")
  .upload("eq-42.jpg", bytes) { upsert = true }
```

Dica

Buckets têm políticas RLS como as tabelas (`storage.objects`).

supabase/functions/relatorio/index.ts

```ts
Deno.serve(async (req) => {
  const { laboratorio, mes } = await req.json();
  // usa a chave secreta, que fica no servidor
  const pdf = await gerarRelatorio(laboratorio, mes);
  return new Response(JSON.stringify({ url: pdf }),
    { headers: { "Content-Type": "application/json" }
    });
});
```

Chamando do app (Dart)

```dart
final r = await db.functions.invoke('relatorio',
    body: {'laboratorio': 'lab-3', 'mes': '2026-10'});
```

Outros provedores

PocketBase e Appwrite

PocketBase num Raspberry Pi

```
$ ./pocketbase serve --http=0.0.0.0:8090
# painel em http://ip-do-pi:8090/_/
```

API REST gerada (HTTP)

```
GET  /api/collections/bancadas/records
     ?filter=(livre=true)
POST /api/collections/reservas/records
POST /api/collections/users/
     auth-with-password
```

PocketBase

Um executável em Go com SQLite, login, regras, arquivos, tempo real e painel. Ótimo para protótipos e redes locais.

Appwrite

Plataforma aberta e auto-hospedável (ou na nuvem Appwrite) com banco, autenticação, arquivos, funções em várias linguagens, mensagens e tempo real. Permissões definidas por linha e por papel.

AWS Amplify

O backend é descrito em TypeScript (Amplify Gen 2) e vira serviços da AWS: Cognito (login), AppSync (GraphQL), DynamoDB e S3. Indicado para quem já está na AWS.

Escolhendo e arquitetando

Matriz de decisão para o LabMonitor

Critério	Peso	Firebase	Supabase	PocketBase
Dados relacionais e relatórios	3	2	5	3
Regras de integridade no banco	3	2	5	3
Tempo real e offline	2	5	4	3
Push, crash e analytics prontos	2	5	2	1
Auto-hospedar no IFMA	2	1	4	5
Custo para a turma	1	4	4	5
Total ponderado		38	**54**	41

Notas ilustrativas de 1 a 5; refaça com os pesos do seu projeto.

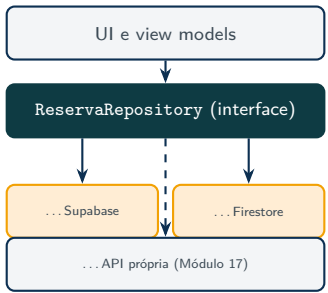
Leitura do resultado

Para reservas e relatórios, o modelo relacional pesa mais. Uma combinação também é válida: **Supabase** para dados e **Firebase** só para push (FCM) e Crashlytics.

Atenção

Discuta os pesos antes das notas, para não ajustar a matriz ao resultado desejado.

Arquitetura que sobrevive à troca de provedor



Regras

- Tipos do SDK (`PostgrestException`, `DocumentSnapshot`) não passam da camada de dados.
- O domínio usa modelos próprios (`Reserva`, `Bancada`).
- Erros do provedor viram `Result` com erros do domínio.

Dica

Com isso, migrar do BaaS para o backend próprio do Módulo 17 muda uma classe por repositório, não o app inteiro.

Fechamento

Resumo

Resumo

- O backend compartilha dados, aplica regras confiáveis e guarda segredos.
- IaaS, PaaS, FaaS, BaaS e SaaS diferem em quanto o provedor cuida por você.
- *Serverless* cobra por uso: alertas de orçamento são obrigatórios.
- Firebase, Supabase, Amplify, Appwrite, PocketBase e Parse têm modelos de dados, licenças e preços diferentes.
- No Supabase, o PostgreSQL com RLS e restrições é o centro da segurança e da integridade.
- Repositórios isolam o provedor e reduzem o *lock-in*; a LGPD guia a escolha da região e do contrato.

Glossário

Termo	Significado
Backend	parte do sistema nos servidores: dados, regras, autenticação, integrações
BaaS	*Backend as a Service*: backend pronto, acessado por SDK e API
Serverless	modelo em que o provedor gerencia e escala servidores; paga-se por uso
Partida a frio	atraso da primeira execução de uma função inativa
Lock-in	dependência que torna cara a troca de fornecedor
RLS	segurança em nível de linha do PostgreSQL, por usuário
JWT	*token* assinado que carrega a identidade do usuário
Restrição de exclusão	regra do PostgreSQL que impede linhas conflitantes (intervalos sobrepostos)
Controlador / operador	quem decide sobre os dados pessoais / quem os trata em seu nome (LGPD)

Questões de revisão

1. Cite três coisas que o backend faz e que o app sozinho não consegue fazer com segurança.
2. Diferencie PaaS, FaaS e BaaS com um exemplo de cada.
3. O que é partida a frio e quando ela incomoda o usuário?
4. Por que a chave pública do Supabase pode ir no app e a secreta não?
5. Explique o que `using` e `with check` fazem numa política RLS.
6. Como a restrição de exclusão evita reservas sobrepostas? Compare com a solução do Firestore.
7. Que práticas reduzem o *lock-in*?
8. Que perguntas de LGPD você faria antes de contratar um BaaS?

Exercícios

Exercício 1 — Modelagem e RLS

Crie as tabelas `laboratorios`, `bancadas` e `reservas` no Supabase, com RLS em que só técnicos (papel guardado em `perfis`) cadastram bancadas.

Exercício 2 — Tempo real

Mostre no app, em tempo real, as reservas do dia de uma bancada, testando com dois aparelhos.

Exercício 3 — Comparação

Implemente a mesma listagem de bancadas no PocketBase e compare esforço, recursos e desempenho com o Supabase.

LabMonitor — Etapa 16

Reservas no Supabase com RLS e restrição de exclusão, atualização em tempo real e fotos no Storage, tudo atrás de `ReservaRepository`.

Para aprofundar

- Supabase: `https://supabase.com/docs`
- RLS no Supabase: `https://supabase.com/docs/guides/database/postgres/row-level-security`
- Appwrite: `https://appwrite.io/docs` PocketBase: `https://pocketbase.io/docs`
- AWS Amplify: `https://docs.amplify.aws`
- Parse Platform: `https://parseplatform.org`
- LGPD (Lei 13.709/2018): `https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm`

Respostas das questões de revisão

1. Compartilhar dados entre usuários, aplicar regras que não podem ser burladas e guardar segredos (também: tarefas com o app fechado).
2. PaaS roda sua aplicação (Cloud Run); FaaS roda funções por evento (Cloud Functions); BaaS entrega banco, login e arquivos prontos (Supabase).
3. É o tempo para iniciar uma função inativa; incomoda em ações interativas raras, como o primeiro toque em "gerar relatório".
4. A pública só permite o que o RLS autoriza para aquele usuário; a secreta ignora o RLS e daria acesso total a quem extraísse o app.
5. `using` define quais linhas existentes o usuário pode ver, alterar ou apagar; `with check` valida as linhas novas ou alteradas.
6. O banco recusa duas linhas com a mesma bancada e intervalos que se sobrepõem, para qualquer cliente. No Firestore, a regra dependia de uma transação feita pelo app.
7. Repositórios que escondem o SDK, modelos de domínio próprios, padrões abertos (SQL, REST, JWT), exportação de dados e possibilidade de auto-hospedagem.
8. Onde ficam os dados (região), se há contrato de operador, como exportar e excluir dados de um usuário, onde ficam logs e cópias.

Fim do Módulo 16

Próximo: Módulo 17 (comum) — Backend próprio: API REST com Dart (Shelf) e Kotlin (Ktor), PostgreSQL, JWT, Docker e implantação.